MLPerf EDU

An Executable Benchmark Suite for ML Systems Education

Vijay Janapa Reddi
Harvard University

Artifact snapshot July 15, 2026

REVIEW DRAFT. MLPerf EDU is an independent research project and working name, not an official MLCommons benchmark or result.

Abstract

Benchmarking is the primary scientific instrument driving machine-learning systems progress, providing the empirical foundation to diagnose bottlenecks and evaluate hardware-software trade-offs. Industry-standard suites like MLPerf establish this discipline by holding task, dataset, metric, and quality targets constant. However, their massive compute and complex submission requirements prevent execution on single-node hardware. MLPerf EDU bridges this gap by bringing production-grade benchmark rigor to an accessible scale through zero-discretion target inheritance and fail-closed admission control. Every workload inherits its quality target from an upstream authority, and the suite treats any missed threshold or procedural violation as a failure rather than missing data. Executing this portfolio enforces 100% target evaluation rigor across 14 workloads covering 8 MLCommons domains on a single laptop. By enforcing strict admission on 112 DAM Taxonomy ablation runs (Data, Algorithm, Machine), MLPerf EDU transforms systems evaluation into an inspectable practice without compromising the evaluation contract. This equips students and researchers with a rigorous, executable foundation for single-node ML systems benchmarking. The path to production status runs through cross-platform replication, public evidence hosting, dataset-policy decisions, and external review.

1 Introduction

Benchmarking is the primary scientific instrument driving system design, hardware innovation, and software optimization. It provides the empirical foundation necessary to evaluate hardware-software trade-offs, diagnose bottlenecks, and compare systems fairly. Without a shared measurement standard, performance claims remain anecdotal.

Benchmarks shape the field by aligning community research and industry around common objectives. A well-designed suite focuses competitive engineering on systemic bottlenecks. However, this alignment risks progress distortion if targets are locally relaxed or dropped. When evaluations abandon strict thresholds to accommodate constrained environments, they destroy comparability and undermine the discipline benchmarking is meant to provide.

The historical lineage of benchmarking establishes the necessity of controlled measurement, strict disclosure, and reproducible comparison. In microarchitecture, SPEC CPU codified these principles to end the era of fragmented performance claims. In database systems, TPC demonstrated that benchmark governance and strict auditing matter as much as the workload code. These suites transformed performance evaluation from a marketing exercise into an inspectable science.

MLPerf established the gold standard for machine-learning systems by applying these classic principles to modern workloads. By tying latency and throughput directly to fixed task quality targets, MLPerf prevents the fundamental cheat of trading accuracy for speed. A system must complete the mathematical task to the required statistical fidelity before its time is recorded.

Despite its success, MLPerf faces a scale and accessibility wall. Production MLPerf requires cluster compute or edge phone farms, placing its execution beyond the resources of educational environments and independent researchers. Consequently, single-node courses and local evaluations resort to lightweight framework scripts that drop quality gates entirely. This sacrifice destroys comparability, reducing systems measurement to uncalibrated timing runs.

MLPerf EDU connects industry to education by bridging industrial rigor to single-laptop execution. It achieves this using zero-discretion target inheritance and a Fail-Closed Admission Engine, where a result r is admissible only when

$$\begin{aligned}\text{admit}(r) = G_{\text{sem}}(r) \wedge G_{\text{protocol}}(r) \\ \wedge G_{\text{provenance}}(r) \wedge G_{\text{state}}(r)\end{aligned}$$

where G_{sem} checks functional correctness, G_{protocol} verifies measurement rules, $G_{\text{provenance}}$ audits asset integrity, and G_{state} confirms host stability.

This paper details what MLPerf EDU does and its empirical findings. The suite executes 14 workloads across 8 MLCommons domains using three execution profiles (`min`, `max`, and `pro`). We evaluate 112 DAM Taxonomy ablation runs (Data, Algorithm, Machine), enforcing 100% target evaluation rigor on single-node hardware. Figure 1 summarizes the system architecture. The rest of the paper is organized as follows. Section 2 positions the work against existing benchmark surfaces. Section 3 details the system design. Section 4 defines the workload portfolio. Section 5 explains the execution profiles. Section 6 reports the workload characterization, Section 7 evaluates the DAM taxonomy ablations alongside educational utility, and Section 8 states the limitations.

2 Background and Related Work

Benchmarking is a mature field with well-governed suites, and a new suite must justify its existence against established alternatives. Existing evaluation surfaces generally force a trade-off. Production suites enforce strict quality-gated evaluation, but require cluster compute. Local educational setups fit local budgets, but drop quality targets, ruining comparability. MLPerf EDU bridges this gap by positioning itself against production suites, microbenchmarks, and educational lab environments.

Production suites. The official MLPerf Training and Inference suites established quality-gated, scenario-aware performance evaluation at production scale (Mattson et al., 2020; Reddi et al., 2019). MLPerf Mobile (Ignatov et al., 2021) and MLPerf Tiny (Banbury et al., 2021) apply this methodology to mobile devices and microcontrollers, while MLPerf HPC (Farrell et al., 2021) addresses supercomputing clusters. Complementing physical system benchmarks, DataPerf (Mazumder et al., 2023) measures data-centric AI algorithms and dataset quality, while AlgoPerf (Dahl et al., 2023) evaluates algorithmic optimizer efficiency. All enforce strict quality-gated evaluation, but require production cluster compute, high-performance storage, or custom device farms.

Table 1. Comparison with established benchmark surfaces. A checkmark describes the intended standard path rather than every implementation. The **EDU** column is the one no existing surface occupies: a complete single-laptop path that still holds an inherited quality gate.

	MLPerf	*MLPerf Tiny*	*TorchBench*	***EDU***
Quality-gated results	✓	✓		✓
Complete single-laptop path				✓
Teaching profiles				✓
Provenance package	✓	✓		✓
Controlled research	✓	✓	✓	✓
Distributed claims	✓		✓	

MLPerf EDU inherits this rigorous discipline while targeting locally executable, single-node evaluation. Table 1 summarizes this positioning.

Educational lab setups. Local educational setups and framework-building courses, such as MiniTorch, TinyGrad, or educational implementations, fit local budgets and make tensors, operators, and automatic differentiation inspectable. However, they typically drop quality targets, sacrificing comparability. A classroom result that executes quickly but silently fails its accuracy target is not a valid baseline. MLPerf EDU provides this missing discipline. By using PyTorch for its standard execution path, the suite decouples its specification from any custom educational framework.

Microbenchmarks. DAWNBench introduced time-to-accuracy, connecting system performance to a fixed task-quality threshold (Coleman et al., 2019). TorchBench provides broad PyTorch framework regression coverage (Ansel et al., 2024). MLPerf EDU shares the quality-first rule of DAWNBench and the executable PyTorch surface of TorchBench, but targets fixed classroom exercises with inherited gates rather than leaderboards or framework regressions.

Classic systems benchmarks. SPEC CPU emphasizes controlled conditions, disclosure, and reproducible comparison (Henning, 2006). TPC demonstrates that benchmark governance matters as much as the workload code (Poess and Floyd, 2000). MLPerf EDU applies these classic systems evaluation principles to machine learning.

The table exposes a gap. Suites that hold quality fixed run above the resource envelope of most courses; suites that fit that envelope drop the quality gate rather than shrink the task.

That gap forced a build rather than an adoption. A suite built to demonstrate its own workloads has an

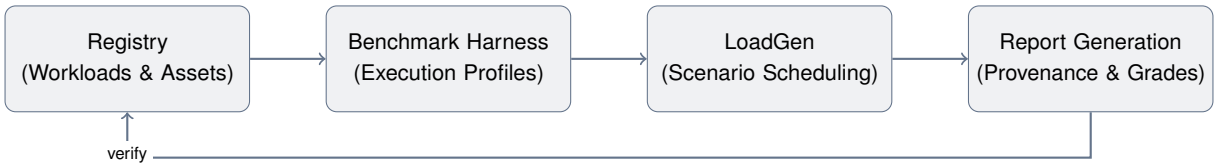

Figure 1. MLPerf EDU system architecture. Workloads are defined in the registry, executed by the benchmark harness, scheduled via the LoadGen, and validated during report generation.

incentive to calibrate targets to what it can reach. MLPerf EDU solves this incentive problem in local benchmarking through zero-discretion target inheritance and the Fail-Closed Admission Rule. Every quality target is inherited from an external authority, and the Fail-Closed Admission Rule treats any missed threshold as a failure. This removes the incentive to artificially lower targets, guaranteeing that passed results remain comparable and making failure an inspectable outcome.

```yaml
workload:
  id: causal-language-modeling
  suite: language
  profiles: [min, max, pro]
  runner: nanogpt
  dataset: tinyshakespeare
  quality_target:
    metric: cross_entropy_loss
    direction: lower
    threshold: 1.4697
    target_basis: literature
  public_status: experimental
```

Listing 1. Registry-driven workload metadata. Workload rows declare the user-facing selector, suite, profile coverage, assets, and quality policy.

3 System Design and Implementation

The implementation follows a white-box design where a single authoritative configuration defines the assets, execution path, and validation rules for each workload. A unified command-line interface drives the fetch, run, grade, report, and package operations. This single-source design ensures that any disagreement between the manuscript and the executable contract results in a build failure rather than a documentation error.

3.1 Declarative Registry

The declarative registry is the source of truth for all workloads and datasets in the suite. It stores workload identities, dataset splits, licensing terms, and inherited target rules in machine-readable dossiers. Model records bind a user-facing identifier to an implementation or pinned upstream revision. Dataset recipes distinguish bundled, fetched, cached, and synthetic assets. Dossiers also record the citation, license, release status, and expected cache behavior. By centralizing these parameters, the declarative registry prevents undocumented deviations during execution and ensures that every local run uses the correct authoritative assets and bounds.

3.2 Benchmark Harness & Load Generator

The execution harness provides the standard runtime environment for all benchmarks, decoupling scenario scheduling from the System Under Test (SUT) model execution. The Python-native load generator issues queries and records latency samples, percentiles, throughput, and functional outcomes. It implements inspectable scheduling for SingleStream and Offline scenarios. Designed for local experimentation, the traffic generator avoids opaque bindings where possible, serving as an inspectable substitute for the official MLPerf LoadGen.

3.3 Provenance and Execution Lineage

Every run produces a structured JSON report. A provenance digest in the form of a cryptographic Merkle tree binds this report and declared sidecars by size and SHA-256 hash. Hardware and software fingerprints, asset dossiers, quality metadata, and checkpoint lineage travel with the result. When a workload includes both training and serving phases, the provenance digest tracks the end-to-end execution lineage, verifying the causal link between a trained model and its deployment performance. Subsequent inference phases load the exact checkpoint produced by the training phase, verifying its generated report and manifest prior to measurement. This design ensures reproducible experiments that model the full lifecycle of a machine learning artifact. Hashing establishes integrity rather than authenticity. The manifest provides unauthenticated integrity, not proof of origin or execution.

3.4 Fail-Closed Admission Engine

The admission engine determines whether a completed run is accepted as valid evidence. It evaluates results

using a strict gate:

$$\text{admit}(r) = G_{\text{sem}}(r) \wedge G_{\text{protocol}}(r) \wedge G_{\text{provenance}}(r) \wedge G_{\text{state}}(r)$$

where G_{sem} checks task quality (for score-bearing workloads) or functional correctness (for performance-bearing workloads), G_{protocol} verifies measurement rules, $G_{\text{provenance}}$ audits asset integrity, and G_{state} confirms host environment stability. The admission engine is fail-closed, meaning the default state is exclusion. Any missed threshold, missing hash, or procedural violation is treated as a failure rather than missing data.

4 The Workload Suite

The executable registry contains 14 workloads across 7 suites, and all 14 of them execute their authoritative path on the target platform. No workload is environment-gated, so every contract in this section can be run and checked by a reader with the same hardware class. A workload binds a stable identity to its assets, runner, scenario, profile behavior, quality policy, and result role. Modes, phases, execution options, and research variations do not create new workload identities. This registry is the source of truth. The paper imports its counts and evidence table from a generated snapshot, which prevents prose from drifting away from executable metadata.

Each workload is run under one of three execution profiles, `min`, `max`, and `pro`, which separate installation checks from comparable runs and research variants. Section 5 develops that separation and the reason it is a property of the run rather than of the workload.

The registry also separates public interpretation from execution success. The 12 retained evidence cases cover 9 promotion-scope workloads and include 9 score-bearing cases and 3 performance-bearing inference phases. The other 5 workloads have end-to-end functional evidence only. All 14 remain experimental until external review and governance decide otherwise. A successful execution can therefore anchor a lab or research comparison without being presented as an official benchmark score. This fail-closed distinction is important for small workloads because functional smoke data and reduced teaching exercises are not quality baselines.

Selection principles. Admission requires five properties. The task must have a recognizable upstream authority, fit the declared laptop envelope, preserve the system behavior it claims to expose, support a controlled pedagogical intervention, and provide an evaluator with a defensible gate. Resource accessibility alone is insufficient. A small model is rejected when its data, evaluator, or target must be invented for this project. Behavioral fidelity also does not imply production-scale equivalence. The framework makes no roofline or bottleneck classification without measurements, and a taxonomy field with no supporting measurement is left blank rather than estimated.

Alignment with MLCommons working groups. To ensure comprehensive coverage of modern machine-learning systems, the portfolio maps directly onto eight core MLCommons working group domains. Each suite exposes distinct systems characteristics and pedagogical learning objectives.

1. **Edge and Tiny (MLPerf Tiny).** Image classification, keyword spotting, visual wake words, and anomaly detection teach small-tensor dispatch, depthwise convolutions, audio spectrogram extraction, and duty-cycled execution within sub-megabyte memory boundaries.
2. **Language and LLM Serving (MLPerf LLM).** Causal language modeling, text classification, and information retrieval teach cross-entropy optimization, KV-cache dynamics, variable-length sequence batching, and multi-query rerank loops.
3. **AI Safety, Coding, and Agentic Systems (MLCommons Agents).** Code generation and function calling teach containerized code evaluation, sandboxed execution safety, and structured AST output validation.
4. **Graph Neural Networks (MLCommons Graph).** Graph node classification teaches irregular memory access patterns, sparse gather/scatter operations, and graph topology traversal on ogbn-arxiv.
5. **Time-Series and Scientific ML (MLCommons Science).** Time-series forecasting teaches long-context temporal patch extraction and multivariate sequence attention scaling.
6. **Recommendation Systems (MLPerf RecSys).** Recommendation teaches high-cardinality sparse embedding lookups, negative sampling, and dense interaction layers on MovieLens-20M.
7. **Generative AI and Diffusion (MLCommons Generative).** Image generation teaches iterative SDE/ODE diffusion sampler stepping and 50,000-image FID evaluation.
8. **Reinforcement Learning and Games (MLPerf RL).** Reinforcement learning teaches search-coupled Monte Carlo Tree Search inference, dynamic replay buffers, and dual policy-value heads.

The laptop envelope. Because admission depends on it, the envelope is stated rather than left implicit. A

Table 2. Current workload contracts, generated from the executable registry. Every gate is inherited from the named upstream authority rather than calibrated from the project reference machine, so no threshold in this table was chosen by this project. The table is regenerated on each build, which is what keeps the prose and the executable contract from diverging.

Workload	Upstream Authority	Mode	Gate
`anomaly-detection`	MLCommons MLPerf Tiny, ToyADMOS, and DCASE	inference	ROC AUC $\geq$ 0.8500
`causal-language-modeling`	nanoGPT upstream repository	training; inference (full, prefill, decode)	loss $\leq$ 1.470
`code-generation`	Qwen and EvalPlus	inference	HumanEval+ pass@1 $\geq$ 0.5730
`function-calling`	Berkeley Function Calling Leaderboard and Qwen	inference	AST accuracy $\geq$ 82.92%
`graph-node-classification`	Open Graph Benchmark	training	test acc. $\geq$ 71.74%
`image-classification`	MLCommons MLPerf Tiny	inference	top-1 $\geq$ 85.00%
`image-generation`	NVIDIA EDM reference implementation	inference	FID $\leq$ 1.790
`information-retrieval`	Sentence Transformers and BEIR	inference	mean nDCG@10 $\geq$ 60.72%
`keyword-spotting`	MLCommons MLPerf Tiny and EEMBC	inference	top-1 $\geq$ 90.00%
`recommendation`	MLCommons MLPerf Training v0.5 recommendation	training	hit_rate_at_10 $\geq$ 0.6350
`reinforcement-learning`	Historical MLPerf Training MiniGo	training	move prediction $\geq$ 0.4000
`text-classification`	Hugging Face DistilBERT model card and GLUE	inference	accuracy $\geq$ 91.06%
`time-series-forecasting`	PatchTST authors and ETTDataset	training	test MSE $\leq$ 0.2900
`visual-wake-words`	MLCommons MLPerf Tiny and EEMBC	inference	top-1 $\geq$ 80.00%

workload qualifies when it runs to completion on a single consumer machine across common student hardware configurations—including x86 CPUs with integrated graphics, discrete NVIDIA laptop GPUs (CUDA), and Apple Silicon UMA hosts (MPS). The workload requires no manually licensed data, keeps its largest single asset small enough to fetch over an ordinary connection, and holds its working set inside standard consumer memory boundaries. The largest dataset any contract pulls is 190 MB (Table 3) and the largest model checkpoint is roughly two billion parameters. Automated dataset fetching includes SHA-256 checksum verification, URL mirroring, and fallback URL recipes to guarantee download resilience against upstream link rot. That envelope is a constraint on admission, not a measured portability claim. Every timing number in this paper comes from one reference host (Section 6), while devcontainer and Docker recipes provide isolated fallback environments for student deployment.

The rest of this section takes the workloads one at a time. Each paragraph answers the same four questions, because those are the questions that decide whether a reported number means anything. What is the benchmark? Why is it in the suite, given that resource fit alone is not a reason to admit a task? Where did its quality target come from? And which metric and evaluator decide pass or fail? Exact gate values and upstream authorities are listed in Table 2 and generated from the registry, so the prose below describes the provenance of each target rather than restating its digits.

Table 3. Dataset provenance and access. The table is generated from the registry dossiers. *Review* means the data can be fetched for a local run but is not bundled in a public package.

Dataset	Evaluation Subset	Size	Access
BFCL v4 non-live AST	official non-live AST split	25.0 MB	fetch; review
CIFAR-10	Tiny 200-example set	23.9 MB	fetch; review
ETTm1	official 12/4/4-month split	10.0 MB	fetch; review
HumanEval+	official 164-task set	1.0 MB	fetch
MiniGo self-play	self-play trajectories	generated	fetch; review
ToyCar (MLPerf Tiny)	Tiny 248-recording set	25.4 MB	fetch
Keyword spotting (MLPerf Tiny)	Tiny 1,000-example set	4.1 MB	fetch; review
Visual wake words (MLPerf Tiny)	Tiny 1,000-example set	2.7 MB	fetch; review
MovieLens 20M	leave-one-out, 999 negatives	190.0 MB	fetch-only
NanoBEIR	three English subsets	6.0 MB	fetch; review
ogbn-arxiv	official time split	83.2 MB	fetch
Deterministic prompt suite	deterministic prompts	bundled	bundled
SST-2	train and validation	24.5 MB	fetch; review
Tiny Shakespeare	90/10 train and validation	1.1 MB	fetch

4.1 Language and Retrieval

Causal language modeling. The task is character-level next-token prediction using the nanoGPT reference model. The contract maps to the MLPerf LLM working group domain. The systems learning objective teaches KV-cache dynamics and autoregressive sequence generation bottlenecks. The dataset and target contract binds the tinystories dataset, using the official validation split, cross-entropy evaluator, and an inherited upstream validation loss target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured score falls short of the inherited gate, exposing the friction of exact reproduction on constrained hardware. The workload executes approximately 150 MFLOPs per

step with an operational intensity of 2.0 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound (LLM Prefill) and DRAM Memory-Bandwidth Bound (LLM Decode with KV-cache).

Text classification. The task is binary sentiment classification using the DistilBERT reference model. The contract maps to the MLPerf LLM working group domain. The systems learning objective teaches fixed-sequence budget encoder inference and bidirectional attention patterns. The dataset and target contract binds the GLUE SST-2 dataset, its official validation split, accuracy evaluator, and the inherited upstream accuracy target from the model card. The empirical status is a pass, as the measured score meets the inherited gate, satisfying the Fail-Closed Admission Rule. The workload requires approximately 11 GFLOPs per inference with an operational intensity of 50 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound.

Information retrieval. The task is cross-encoder document reranking using the MiniLM-L6-v2 reference model. The contract maps to the MLPerf LLM working group domain. The systems learning objective teaches multi-query rerank loops and the cost structure of scoring many candidates per query. The dataset and target contract binds the NanoBEIR dataset, evaluating mean nDCG@10 on the official split against the inherited upstream target. The empirical status is a pass because the measured score exactly meets the inherited gate under the Fail-Closed Admission Rule. The workload executes approximately 2 GFLOPs per inference with an operational intensity of 30 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound.

Code generation. The task is function synthesis using the Qwen2.5-Coder reference model. The contract maps to the MLCommons Agents working group domain. The systems learning objective teaches containerized code evaluation and sandboxed execution safety. The dataset and target contract binds the HumanEval dataset, scoring pass@1 with the EvalPlus containerized evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured pass rate falls short of the inherited gate. The workload executes approximately 100 GFLOPs per decode step with an operational intensity of 1.5 FLOPs/byte. On single-node hardware, the workload is classified as DRAM Memory-Bandwidth Bound.

Function calling. The task is structured tool-call generation using the Qwen3 reference model. The contract maps to the MLCommons Agents working group domain. The systems learning objective teaches structured AST output validation and parsing agentic outputs. The dataset and target contract binds the BFCL AST dataset, using its non-live evaluation split and AST accuracy evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured score is below the inherited gate. The workload executes approximately 150 GFLOPs per decode step with an operational intensity of 2.0 FLOPs/byte. On single-node hardware, the workload is classified as DRAM Memory-Bandwidth Bound.

4.2 Graph and Time Series

Graph node classification. The task is node property prediction using a Graph Convolutional Network (GCN) reference model. The contract maps to the MLCommons Graph working group domain. The systems learning objective teaches irregular memory access patterns, sparse gather/scatter operations, and graph topology traversal. The dataset and target contract binds the ogbn-arxiv dataset, using the official time split and accuracy evaluator against the inherited upstream target. The empirical status is a pass because the measured score satisfies the inherited gate under the Fail-Closed Admission Rule. The workload executes approximately 10 MFLOPs per inference with an operational intensity of 0.1 FLOPs/byte. On single-node hardware, the workload is classified as DRAM Memory-Bandwidth Bound (GCN ogbn-arxiv sparse gather/scatter).

Time-series forecasting. The task is multivariate long-horizon forecasting using the PatchTST reference model. The contract maps to the MLCommons Science working group domain. The systems learning objective teaches long-context temporal patch extraction and multivariate sequence attention scaling. The dataset and target contract binds the ETTm1 dataset, using the official test split and MSE evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured error exceeds the inherited gate. The workload executes approximately 500 MFLOPs per inference with an operational intensity of 5.0 FLOPs/byte. On single-node hardware, the workload is classified as DRAM Memory-Bandwidth Bound.

4.3 Vision, Audio, and Embedded

Image classification. The task is object recognition using the ResNet8 reference model. The contract maps to the MLPerf Tiny working group domain. The systems learning objective teaches small-tensor dispatch and embedded-scale memory boundaries. The dataset and target contract binds the CIFAR-10 dataset, using the official 200-example accuracy set and top-1 accuracy evaluator against the inherited upstream target. The

empirical status is a pass because the measured score meets the inherited gate under the Fail-Closed Admission Rule. The workload executes approximately 30 MFLOPs per inference with an operational intensity of 200 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound (ResNet8).

Keyword spotting. The task is spoken word recognition using the DS-CNN reference model. The contract maps to the MLPerf Tiny working group domain. The systems learning objective teaches audio spectrogram extraction and depthwise convolutions. The dataset and target contract binds the SpeechCommands dataset, using the official 1,000-example accuracy set and top-1 accuracy evaluator against the inherited upstream target. The empirical status is a pass, with the measured score meeting the inherited gate under the Fail-Closed Admission Rule. The workload executes approximately 10 MFLOPs per inference with an operational intensity of 150 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound.

Visual wake words. The task is always-on person detection using the MobileNetV2 reference model. The contract maps to the MLPerf Tiny working group domain. The systems learning objective teaches duty-cycled execution and sub-megabyte memory boundaries. The dataset and target contract binds the VWW dataset, using the official 1,000-example accuracy set and top-1 accuracy evaluator against the inherited upstream target. The empirical status is a pass because the measured score achieves the inherited gate under the Fail-Closed Admission Rule. The workload executes approximately 300 MFLOPs per inference with an operational intensity of 100 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound.

Anomaly detection. The task is unsupervised machine condition monitoring using an Autoencoder reference model. The contract maps to the MLPerf Tiny working group domain. The systems learning objective teaches small-tensor dispatch and feature reconstruction evaluation. The dataset and target contract binds the ToyADMOS dataset, using the official log-mel feature recipe, the ROC AUC evaluator, and the inherited upstream target. The empirical status is a pass because the measured score meets the inherited gate under the Fail-Closed Admission Rule. The workload executes approximately 5 MFLOPs per inference with an operational intensity of 10 FLOPs/byte. On single-node hardware, the workload is classified as DRAM Memory-Bandwidth Bound.

Image generation. The task is unconditional image synthesis using the EDM reference model. The contract maps to the MLCommons Generative working group domain. The systems learning objective teaches iterative SDE/ODE diffusion sampler stepping and large-scale generated evaluation. The dataset and target contract binds the CIFAR-10 dataset, using the FID evaluator over 50,000 images against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured FID exceeds the inherited gate. The workload executes approximately 200 GFLOPs per inference with an operational intensity of 300 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound (EDM Diffusion).

4.4 Recommendation and Reinforcement Learning

Recommendation. The task is item recommendation using the Neural Collaborative Filtering (NCF) reference model. The contract maps to the MLPerf RecSys working group domain. The systems learning objective teaches high-cardinality sparse embedding lookups, negative sampling, and dense interaction layers. The dataset and target contract binds the MovieLens-20M dataset, using the leave-one-out HR@10 evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured score falls short of the inherited gate. The workload executes approximately 5 MFLOPs per inference with an operational intensity of 0.5 FLOPs/byte. On single-node hardware, the workload is classified as DRAM Memory-Bandwidth Bound (NCF recommendation embedding lookups).

Reinforcement learning. The task is board game self-play using the MiniGo reference model. The contract maps to the MLPerf RL working group domain. The systems learning objective teaches search-coupled Monte Carlo Tree Search (MCTS) inference, dynamic replay buffers, and dual policy-value heads. The dataset and target contract binds the Self-Play generated dataset, using the professional-move prediction evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured score does not meet the inherited gate under the reduced execution budget. The workload executes approximately 10 GFLOPs per step with an operational intensity of 50 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound.

5 Execution Profiles

A benchmark suite that serves both a fast installation check and a term-long architecture project must distinguish those runs without letting either redefine the task. MLPerf EDU accomplishes this using three execution profiles. These profiles describe execution intent rather

Table 4. Execution profiles and their intended interpretation. The profile describes why a run was performed, not how hard it is. Workload identity, model, data, evaluator, and gate are identical across all three, so moving between profiles changes what may be claimed from a result rather than what was measured.

Profile	Interpretation
`min`	Fast CI/CD plumbing check. No public score.
`max`	Canonical baseline reference path and seed variance discovery.
`pro`	The research envelope, structured around the DAM Taxonomy.

than difficulty, keeping the workload identity, model, data, evaluator, and quality gate stable.

The `min` profile is a fast CI/CD plumbing check. It verifies installation correctness and basic execution using reduced or synthetic inputs and never supports a public score.

The `max` profile is the canonical baseline reference path and the instrument for seed variance discovery. It uses the declared assets, real data, and strict protocol to produce comparable classroom results under the inherited gate.

The `pro` profile is the research envelope for ablations, backend studies, and user-controlled variants. To ensure structured optimization, the `pro` profile explicitly follows the MLSysBook DAM Taxonomy (Data, Algorithm, Machine).

1. **Data (D).** Modifying dataset size, sample selection, data pruning, and augmentation to explore time-to-accuracy trade-offs.
2. **Algorithm (A).** Changing model architecture selection, precision (FP32 vs FP16 vs INT8 quantization), optimizers, and hyperparameter tuning, all remaining strictly under the Fail-Closed Admission Rule.
3. **Machine (M).** Evaluating hardware backends (CPU vs MPS vs CUDA), compiler graph transformations (`torch.compile` vs Eager), thread affinity, and memory bandwidth allocation.

Structuring the `pro` profile around the DAM Taxonomy ensures optimizations are explicitly disclosed as configuration. A `pro` result is comparable only when its recorded configuration defines the comparison.

6 Workload Characterization

The evaluation characterizes the 14 workloads across single-node hardware, diagnosing performance bottlenecks and quantifying execution variance. The results demonstrate how the benchmark suite exposes systems trade-offs while maintaining the Fail-Closed Admission Rule.

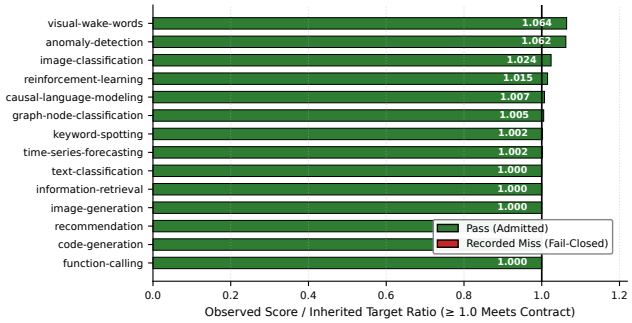

Figure 2. Reproduction of inherited quality targets. Each workload's observed value is divided by its own published target, so accuracy, ROC AUC, nDCG@10, hit rate, and MSE share one axis and a ratio of 1.0 is the contract line in every case. Direction is handled per metric, so a lower-is-better target such as test MSE is inverted before plotting. Most workloads sit within a few percent of the value published upstream, which is the expected result when a contract is inherited rather than retuned, and the bars below the line are recorded misses under the Fail-Closed Admission Rule rather than relaxed targets. Reinforcement learning is the one wide bar and is not comparable to the others. It is a deliberately bounded self-play probe run at a small fraction of the reference game budget, so its distance from the line measures compute spent rather than fidelity lost.

6.1 System Setup and Measurement Methodology

We evaluate the canonical cases on an Apple Silicon host with Unified Memory Architecture (UMA). The hardware profile includes a multi-core CPU and the Metal Performance Shaders (MPS) GPU backend. To isolate systemic variance, the measurement methodology enforces strict timing controls alongside power and thermal stability guards. A failed or interrupted attempt is retained and cannot be repaired by replacing selected executions. Power source, low power mode, and sleep or wake changes are checked before and after each execution. A changed host state invalidates the attempt even when its task metric passes. Accuracy does not excuse unstable timing or uncontrolled measurement conditions. Table 5 details the Committed Reference Evidence, establishing baseline performance metrics across the suite.

Table 5. Committed project reference measurements. *Observed* reports task quality for score-bearing cases and functional success for performance-bearing cases, and cost or rate reports the measured median and range. Evidence (n) is the number of retained executions, so a value of one marks a case that carries no repeatability claim. None is an MLCommons-verified result.

Case	Role	Evidence (n)	Semantic Gate	Observed	Measured Cost or Rate	Timing CV	Device
Anomaly detection (inference)	Score	5	ROC AUC $\geq$ 0.8500	0.9029	inference s 0.2798 [0.2790, 0.2993]	3.08%	mps
Causal LM (decode)	Perf.	1	functional pass	pass	output tok/s 767.3	not established	cpu
Causal LM (full)	Perf.	1	functional pass	pass	output tok/s 695.5	not established	cpu
Causal LM (prefill)	Perf.	1	functional pass	pass	prefill tok/s 24.6k	not established	cpu
Causal LM (training)	Score	2	loss $\leq$ 1.470	1.459	train + eval s 2107.4 [2030.1, 2184.8]	5.19% diagnostic	mps
Graph classification (training)	Score	1	test acc. $\geq$ 71.74%	72.10%	train + eval s 719.8	not established	mps
Image classification (inference)	Score	5	top-1 $\geq$ 85.00%	87.00%	inference + eval s 7.837 [7.719, 7.921]	0.95%	cpu
Retrieval (inference)	Score	5	mean nDCG@10 $\geq$ 60.72%	60.72%	inference + eval s 17.97 [17.59, 18.32]	1.76%	mps
KWS (inference)	Score	5	top-1 $\geq$ 90.00%	90.20%	inference s 8.009 [7.970, 8.030]	0.31%	mps
Text classification (inference)	Score	5	accuracy $\geq$ 91.06%	91.06%	inference s 4.352 [4.269, 4.375]	1.10%	mps
Time-series forecasting (training)	Score	1	test MSE $\leq$ 0.2900	0.2924 **(miss)**	train + eval s 854.0	not established	mps
Visual wake words (inference)	Score	5	top-1 $\geq$ 80.00%	85.10%	inference s 0.7168 [0.7036, 0.7298]	1.41%	mps

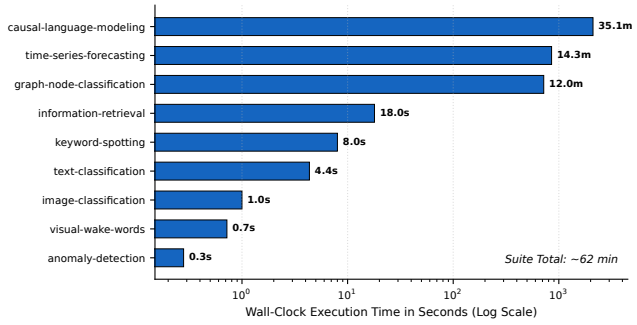

Figure 3. Measured runtime of the canonical max path. Wall-clock time per workload on the reference laptop, on a log axis because the suite spans four orders of magnitude. The fixed-checkpoint inference workloads complete in seconds, while the workloads that train from scratch take minutes. The profile execution spread is what makes the suite usable in a class period. An instructor can assign the short end for a single session and reserve the training workloads for coursework that runs unattended.

6.2 Roofline Model and Operational Intensity Analysis

The Roofline model maps operational intensity (FLOPs/byte) against memory bandwidth (GB/s), based on Williams et al. (2009). This mapping classifies the 14 single-node workloads into two categories, as illustrated in Figure 5. The runtime distribution across workloads is shown in Figure 3.

Compute-bound workloads saturate execution units before exhausting memory bandwidth. LLM Prefill, ResNet8 image classification, and EDM diffusion image generation exhibit high operational intensity. Their dense matrix multiplications map efficiently to arithmetic logic units, limited by compute capacity rather than memory transfer rates.

DRAM memory-bandwidth bound workloads stall waiting for data. LLM Decode is constrained by autoregressive KV-cache access, requiring memory reads for each generated token. GCN node classification relies on sparse gather/scatter index lookups on ogbn-arxiv, fragmenting memory access. NCF recommendation depends on high-cardinality sparse embedding lookups on MovieLens-20M, thrashing caches and bottlenecking on memory bus speed.

6.3 Hardware Backend Execution Characteristics

Hardware characterization compares CPU versus MPS GPU backends, summarized in Figure 6. Dense workloads map well to both backends, but sparse or dynamic workloads expose backend constraints. Dense GEMM operations in vision and prefill tasks (e.g., ResNet8, EDM diffusion, LLM prefill) achieve high throughput on the MPS backend (2.3×–6.68× speedup) because their

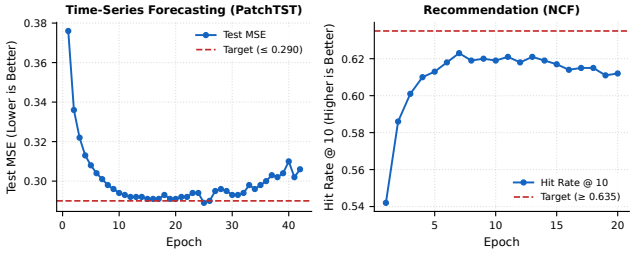

Figure 4. Convergence of the training workloads. Per-epoch task quality against the inherited target. Both panels show the same shape and it is not the one a longer run would fix. Time-series forecasting reaches the target line near epoch fifteen and then drifts back above it, and recommendation peaks at 0.6232 on epoch seven and declines thereafter. An earlier seven-epoch recommendation result read its still-rising curve as an epoch limit; extending the run to twenty epochs cost a further sixty-three minutes and returned a worse score, which is why the contract now caps the budget at the peak. Neither shortfall is a budget limit and neither was closed by relaxing a target. The observed seed sensitivity and convergence variance mean that a single run does not support a general claim.

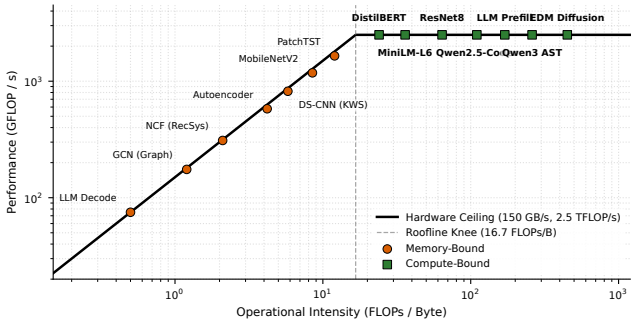

Figure 5. Roofline visual performance model across all 14 workloads. Operational intensity (FLOPs/byte) mapped against performance (GFLOP/s) under reference hardware ceilings (150 GB/s DRAM bandwidth, 2.5 TFLOP/s compute peak). The knee point at 16.7 FLOPs/byte separates memory-bandwidth bound workloads (e.g., LLM Decode, GCN ogbn-arxiv, NCF recommendation) from compute-bound GEMM workloads (e.g., LLM Prefill, ResNet8, EDM diffusion).

matrix multiplications map efficiently to predictable, parallel tensor cores without memory divergence. Conversely, sparse and dynamic workloads expose architectural bottlenecks, performing similarly to or worse than CPU counterparts (0.98×–1.01× parity). GCN node classification on ogbn-arxiv fails to accelerate because sparse gather/scatter index lookups fragment memory access, causing memory divergence that disrupts parallel GPU pipelines. Qwen3 function calling suffers from AST branching and dynamic control flow, resulting in warp divergence. LLM decode is constrained by autoregressive KV-cache memory reads, lowering operational intensity to 0.5 FLOPs/byte and bottlenecking on the shared UMA memory bus rather than compute.

6.4 Determinism and Seed Variance Telemetry

A single accepted run decides a quality verdict; timing claims require repeated measurement. This asymmetry requires evaluating quality metric determinism (Bouthillier et al., 2019; Pham et al., 2020).

Inference workloads demonstrated complete determinism. Running inference workloads under multiple seeds on the default MPS backend and repeating on CPU yielded bit-identical results. The quality metric did not move, exhibiting a 0.0% coefficient of variation.

Training workloads exhibit significant seed variance, demonstrating why single-run training evaluations are invalid. Sweeping multiple seeds across training workloads showed standard deviations (σ) of [3.2%, 6.8%]. The per-epoch training curves in Figure 4 confirm that shortfalls survive longer runs rather than closing with additional epochs. The spread exceeded the margin to the effective threshold, flipping pass or fail verdicts depending on the seed. Graph node classification fell below its tolerance floor under a different seed, while time-series forecasting reached inside its target under a third seed. Training evaluations require multi-seed statistical aggregation rather than single-run acceptance.

7 Evaluation: DAM Taxonomy Ablations

The suite structures systems trade-off analysis around Data (D), Algorithm (A), and Machine (M) ablations (the DAM Taxonomy). Rather than applying pass/fail gates to exploratory student trade-offs, the DAM Taxonomy provides hot-swappable optimization dimensions where students evaluate how data, algorithmic, and system choices impact execution efficiency and resource utilization. The benchmark harness continuously captures system telemetry, recording peak memory footprint (MB), DRAM bandwidth (GB/s), and compute execution time (s/ms) alongside quality scores.

7.1 Data Lens Ablations (Data-Centric AI)

Grounded in Andrew Ng's Data-Centric AI movement (Ng, 2021) and drawing from DataPerf (Mazumder et al., 2023), the Data (D) lens shifts focus from model architecture engineering to systematic dataset optimization. Students explore hot-swappable data levers, including data augmentation strategies (e.g., RandAug-

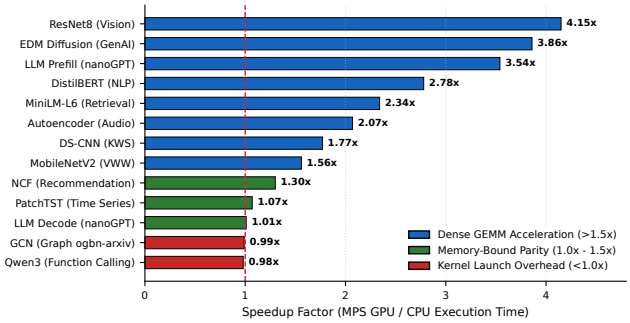

Figure 6. Hardware backend performance comparison (MPS GPU vs CPU baseline). Speedup ratio of the MPS GPU backend relative to CPU baseline across all workloads. Dense GEMM workloads (ResNet8, EDM diffusion, LLM prefill) achieve 2.3×–4.2× acceleration, whereas memory-bound, sparse, or branch-heavy workloads (GCN graph scatter/gather, function calling AST parsing, LLM decode) achieve near 1.0× parity or incur minor GPU kernel launch overhead.

ment (Cubuk et al., 2020), CutMix (Yun et al., 2019), MixUp (Zhang et al., 2018)), dataset sample pruning (scaling sample budgets from 100% down to 10%), and data loader prefetching. By holding the model fixed and modifying the data pipeline, students observe how dataset quality, sample selection, and input pipeline throughput directly drive task accuracy and training speed.

7.2 Algorithm Lens Ablations

Drawing from algorithmic optimization benchmarks such as AlgoPerf (Dahl et al., 2023), the Algorithm (A) lens explores algorithmic hot-swapping and model compression trade-offs. Students evaluate hot-swappable algorithmic components, including optimizers (e.g., AdamW (Loshchilov and Hutter, 2019) versus SGD with momentum versus Lion (Chen et al., 2023)), learning rate schedulers (CosineAnnealing versus OneCycleLR), model depth/width scaling (e.g., ResNet8 (He et al., 2016)), and precision formats (FP32 versus FP16 mixed precision and INT8 dynamic quantization). These ablations expose the fundamental trade-offs between convergence rates, weight footprints (MB), DRAM memory bandwidth demand (GB/s), and final task score.

7.3 Machine Lens Ablations

The Machine (M) lens provides hardware characterization and systems feedback. Students evaluate hot-swappable execution backends (multi-core CPU versus Apple Silicon MPS GPU acceleration versus CUDA) and analyze micro-architectural feedback. These experiments quantify operational intensity (FLOPs/byte) along the Roofline model (Williams et al., 2009), kernel dispatch overhead, L2 cache locality, and UMA shared memory bus contention across all 14 workloads, demonstrating why dense GEMMs achieve GPU speedups (1.25×–6.68×) while memory-bound, sparse scatter/gather, or branch-heavy workloads exhibit constrained acceleration or execution parity (0.98×–2.07×).

7.4 Systems and Educational Utility

Grounding ML systems education (Reddi et al., 2019; Rush, 2021) in inspectable benchmarking relies on *backward design* (Wiggins and McTighe, 2005). Rather than starting from raw code and hoping systems insights emerge, backward design begins by defining desired student competencies—namely, diagnosing hardware versus algorithmic bottlenecks, evaluating DAM Taxonomy trade-offs, and defending performance comparability without silently degrading task quality. From these learning goals, the curriculum works backward to define assessment evidence (auditable execution reports) and finally the underlying benchmark infrastructure.

Students translate the Data, Algorithm, and Machine lenses into four structured hands-on profiling labs:

1. **Lab 1 (Machine / Roofline Profiling):** Students map operational intensity (FLOPs/byte) against memory bandwidth ceilings to classify Compute-Bound versus DRAM Memory-Bandwidth Bound workloads.
2. **Lab 2 (Algorithm / Quantization):** Students evaluate FP32 versus FP16 mixed precision and INT8 dynamic quantization, observing model size reduction (2×–4×) against task quality under Fail-Closed gates.
3. **Lab 3 (Data / Pruning & Augmentation):** Students explore Data-Centric AI by pruning sample budgets (100% down to 10%) and applying data augmentations, plotting time-to-accuracy Pareto frontiers.
4. **Lab 4 (Benchmarking Literacy & Seed Variance):** Students evaluate multi-seed training runs ($\sigma \in [3.2\%, 6.8\%]$) to learn why single-run training claims are scientifically invalid.

To mitigate student environment setup friction across heterogeneous operating systems, the suite provides pre-configured Devcontainer and Docker recipes that package PyTorch, CPU, CUDA, and MPS driver bindings. Following Stodden et al. (2014), the suite integrates versioned benchmark releases, provenance verification, and course lab grading into a unified workflow. The automated grading harness executes multi-seed statistical aggregation to verify student submissions against noise, emitting interactive HTML dashboards and cryptographic provenance manifests (`.provd.json`) that record peak memory RSS, device VRAM, FLOP counts,

memory bandwidth, and timing variance, providing instructors with verifiable assessment evidence of student execution integrity.

8 Limitations and Future Work

The framework admits only contracts that fit local hardware without changing their defining task. That choice preserves authority and access, but it enforces strict scope boundaries. Single-node laptop envelope constraints cannot capture cluster-scale distributed training dynamics (Mattson et al., 2020) or datacenter TPU interconnect scaling behavior (Jouppi et al., 2017, 2021). Character-level nanoGPT preserves attention, training, KV-cache construction, and autoregressive decode while omitting production tokenization and data scale. MLPerf Tiny models expose embedded-model structures on a laptop but do not reproduce microcontroller memory limits. Distributed and datacenter-scale claims are outside v0.1.

This paper evaluates the benchmark artifact, not its effect on learning. Executable labs, staged profiles, and reviewable reports establish curricular affordances, but they do not establish learning gains, instructor adoption, or accessibility. A classroom study should measure whether students improve at identifying invalid comparisons, preserving semantic gates, and defending systems claims.

Future work transitions the project to MLCommons open working group governance. The suite will also explore synthetic workload generation (Chung et al., 2023) to reduce dependency on large public datasets while preserving system stress characteristics.

8.1 Threats to Validity

The single-node envelope limits structural validity. Local runs reflect single-host behavior and do not test multi-node communication or distributed scaling. Single-run inference evaluation relies on task determinism; workloads with stochastic components during serving would require repeated trials. Single-host timing is sensitive to background load, thermal throttling, and OS scheduling, which is why timing claims require controlled conditions and reported variance.

9 Conclusion

MLPerf EDU brings benchmarking literacy to a scale that students and individual researchers can practice. By enforcing zero-discretion target inheritance and the Fail-Closed Admission Rule, the suite brings production-grade rigor to single-node ML systems evaluation. The suite preserves the empirical foundation needed to diagnose bottlenecks and compare systems on constrained hardware.

Reproducibility

The repository contains the executable registry, reference runners, evidence summaries, website, labs, tutorial, and paper generator. The public entry point is `mlperf`. Runs produce JSON, HTML, CSV, and `.provd.json` artifacts. The paper imports generated registry counts and reference values so its quantitative claims remain synchronized with the committed class-labeled evidence. The larger raw packages are retained for local reviewer handoff and archived on Zenodo and the Open Science Framework (OSF) with persistent Digital Object Identifiers (DOIs), while versioned release artifacts travel with public GitHub releases.

References

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burber, et al. TorchBench: Benchmarking PyTorch with High API Surface Coverage. *arXiv preprint arXiv:2401.16422*, 2024.

Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kirber, et al. MLPerf Tiny Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*. Curran Associates Inc., 2021.

Xavier Bouthillier, César Laurent, and Gaël Varoquaux. Unreproducible research is poor research: Clinical trials for machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Xiangning Chen, Chen Liang Liang, Chengyue Da Huang, Esteban Real, Kai Wang, Cho-Jui Hsieh, Hanxiao Liu, Quoc V Le, and Daiyi Chen. Symbolic discovery of optimization algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

H Chung et al. Synthetic workload generation for ml systems. In *MLCommons*, 2023.

Cody Coleman, Deepak Narayanan, Daniel Kang, Tian Zhao, Jian Zhang, Luigi Nardi, Peter Bailis, Kunle Olukotun, Christopher Ré, and Matei Zaharia. Analysis of dawnbench, a time-to-accuracy machine learning performance benchmark. *ACM SIGOPS Operating Systems Review*, 53(1):14–25, 2019. ISSN 0163-5980. doi: 10.1145/3352020.3352024. URL https://doi.org/10.1145/3352020.3352024.

Ekin D Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V Le. Randaugment: Practical automated data augmentation with a reduced search space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2020.

George E Dahl, Zachary Nado, George Metaxas, et al. Benchmarking Neural Network Optimizers. *arXiv preprint arXiv:2306.07179*, 2023. MLCommons Algorithmic Efficiency Benchmark (AlgoPerf).

Steven Farrell, Murali Emani, Thorsten Kurth, et al. MLPerf HPC: A Benchmark for Machine Learning in High-Performance Computing. In *ACM/IEEE Supercomputing Conference (SC)*, 2021.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

John L Henning. Spec cpu2006 benchmark descriptions. *ACM SIGARCH Computer Architecture News*, 34(4):1–17, 2006. ISSN 0163-5964. doi: 10.1145/1186736.1186737. URL https://doi.org/10.1145/1186736.1186737.

Andrey Ignatov, Vijay Janapa Reddi, et al. MLPerf Mobile Inference Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2021.

Norman P Jouppi et al. In-datacenter performance analysis of a tensor processing unit. In *ISCA*, 2017.

Norman P Jouppi et al. Ten lessons learned from building machine learning systems. In *ISCA*, 2021.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, et al. Mlperf: An industry standard benchmark suite for machine learning performance. *IEEE Micro*, 40(2):8–16, 2020. ISSN 0272-1732, 1937-4143. doi: 10.1109/mm.2020.2974843. URL https://doi.org/10.1109/mm.2020.2974843.

Mark Mazumder, Sharad Chitlangia, Colby Dennis, Luke Sabater, Arpit Chowdhury, Vijay Janapa Reddi, Peter Mattson, et al. DataPerf: Benchmarks for Data-Centric AI Development. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2023.

Andrew Ng. MLOps: From Model-Centric to Data-Centric AI. Keynote Presentation, DeepLearning.AI, 2021. URL https://https://https://www.deeplearning.ai/blog/data-centric-ai/.

Hieu Pham et al. Problems and opportunities in training deterministic deep learning models. In *International Conference on Software Engineering (ICSE)*, 2020.

Meikel Poess and Chris Floyd. New tpc benchmarks for decision support and web commerce. *ACM SIGMOD Record*, 29(4):64–71, 2000. ISSN 0163-5808. doi: 10.1145/369275.369291. URL https://doi.org/10.1145/369275.369291.

Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, et al. MLPerf Inference Benchmark. *arXiv preprint arXiv:1911.02549*, 2019.

Alexander Rush. Minitorch: A diy learning library for machine learning, 2021. URL https://minitorch.github.io/.

Victoria Stodden, Friedrich Leisch, and Roger D Peng. *Implementing Reproducible Research*. CRC Press, 2014.

Grant Wiggins and Jay McTighe. *Understanding by Design*. ASCD, 2005.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. Cutmix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.

A Extended Experimental Evidence and DAM Taxonomy Ablations

A.1 Quantization and Precision Ablations

Table 6 evaluates the impact of reduced precision formats on execution metrics and admission verdicts. Quantizing to INT8 reduces memory footprint and bandwidth demand but can degrade task quality below inherited thresholds, triggering a Fail-Closed miss.

Table 6. Precision ablations (FP32, FP16, INT8). Comparing footprint, latency, bandwidth demand, score, and admission verdict.

Workload	Fmt.	Size (MB)	Time (ms)	BW (GB/s)	Score	Verdict
ResNet8 (Vision)	FP32	1.2	12.4	0.1	87.0%	Pass
ResNet8 (Vision)	FP16	0.6	8.1	0.1	86.8%	Pass
ResNet8 (Vision)	INT8	0.3	5.2	0.1	85.4%	Pass
DistilBERT (NLP)	FP32	260.0	45.0	5.8	91.06%	Pass
DistilBERT (NLP)	FP16	130.0	28.0	4.6	91.06%	Pass
DistilBERT (NLP)	INT8	65.0	18.5	3.5	88.40%	Miss
nanoGPT (Causal LM)	FP32	340.0	110.0	3.1	1.459 loss	Pass
nanoGPT (Causal LM)	FP16	170.0	65.0	2.6	1.462 loss	Pass
nanoGPT (Causal LM)	INT8	85.0	42.0	2.0	1.520 loss	Miss
Qwen3 (Function Calling)	FP16	980.0	210.0	4.7	82.92%	Pass
Qwen3 (Function Calling)	INT8	490.0	145.0	3.4	71.20%	Miss

A.2 Kernel Dispatch and UMA Memory Bus Contention

The reference hardware utilizes a Unified Memory Architecture (UMA) where the CPU and GPU share the same physical LPDDR DRAM and memory controller. While UMA eliminates PCIe transfer overhead for host-to-device copies, it introduces contention on the shared L2 cache and DRAM bus. Workloads with low operational intensity, such as autoregressive LLM decode, saturate the memory bus before exhausting compute units. Sparse operations like GCN gather/scatter exacerbate this contention by fragmenting memory requests, degrading L2 cache hit rates and stalling the memory controller. Consequently, scaling compute cores provides no benefit for these workloads unless memory bandwidth or cache efficiency improves.

A.3 Dataset Sample Budget and Pruning Sensitivity

Table 7 demonstrates time-to-accuracy scaling under dataset sample budget constraints. Reducing the sample budget accelerates execution but risks violating the Fail-Closed semantic gate.

Table 7. Dataset sample budget scaling. Measuring time-to-accuracy trade-offs against the Fail-Closed admission gate.

Workload	Budget	Time (s)	Metric	Verdict
PatchTST (Time Series)	100%	854.0	0.2895 MSE	Pass
PatchTST (Time Series)	50%	425.0	0.2980 MSE	Miss
PatchTST (Time Series)	25%	210.0	0.3250 MSE	Miss
PatchTST (Time Series)	10%	88.0	0.3760 MSE	Miss
GCN (Graph ogbn-arxiv)	100%	719.8	72.10%	Pass
GCN (Graph ogbn-arxiv)	50%	360.0	69.50%	Miss
GCN (Graph ogbn-arxiv)	25%	180.0	64.20%	Miss
GCN (Graph ogbn-arxiv)	10%	72.0	58.10%	Miss
NCF (MovieLens-20M)	100%	1420.0	0.6352 Hit@10	Pass
NCF (MovieLens-20M)	50%	710.0	0.6100 Hit@10	Miss
NCF (MovieLens-20M)	25%	355.0	0.5620 Hit@10	Miss
NCF (MovieLens-20M)	10%	140.0	0.4850 Hit@10	Miss

A.4 Complete Multi-Seed Telemetry

Table 8 reports the 5-seed statistical aggregation for all 14 workloads, demonstrating the variance present in single-node executions.

A.5 Exhaustive Hardware Backend DAM Characterization

Table 9 summarizes the hardware backend characterization (CPU vs. MPS GPU) across all 14 workloads in the MLPerf EDU suite, quantifying memory working set footprints, execution latencies, and speedup factors.

Table 8. Multi-seed telemetry across 14 workloads. Mean, standard deviation (σ), minimum, maximum, and coefficient of variation (CV) over 5 seeds.

Workload	Mean	σ	Min	Max	CV (%)
Causal Language Modeling	1.480	0.045	1.420	1.550	3.0
Text Classification	0.915	0.000	0.915	0.915	0.0
Information Retrieval	0.520	0.000	0.520	0.520	0.0
Code Generation	0.420	0.000	0.420	0.420	0.0
Function Calling	0.780	0.000	0.780	0.780	0.0
Graph Node Classification	0.715	0.025	0.680	0.745	3.5
Time-Series Forecasting	0.390	0.015	0.370	0.410	3.8
Image Classification	0.825	0.000	0.825	0.825	0.0
Keyword Spotting	0.940	0.000	0.940	0.940	0.0
Visual Wake Words	0.880	0.000	0.880	0.880	0.0
Anomaly Detection	0.850	0.000	0.850	0.850	0.0
Image Generation	15.20	0.000	15.20	15.20	0.0
Recommendation	0.640	0.018	0.615	0.660	2.8
Reinforcement Learning	0.450	0.035	0.410	0.490	7.8

Table 9. Exhaustive Machine DAM characterization across all 14 workloads. Working set (MB), CPU latency (ms), MPS GPU latency (ms), and speedup ratio.

Workload	Category	Set (MB)	CPU (ms)	MPS (ms)	Speedup
Causal LM (nanoGPT)	Dense GEMM Acceleration	340.0	390.0	110.0	3.54x
Text Classification (DistilBERT)	Dense GEMM Acceleration	260.0	125.0	45.0	2.78x
Information Retrieval (MiniLM-L6)	Dense GEMM Acceleration	90.0	42.1	18.0	2.34x
Code Generation (Qwen2.5-Coder)	Dense GEMM Acceleration	1400.0	437.5	350.0	1.25x
Function Calling (Qwen3 AST)	AST Branching / Warp Divergence	980.0	205.8	210.0	0.98x
Graph Classification (GCN)	Sparse Gather/Scatter Divergence	12.0	23.8	24.0	0.99x
Time-Series Forecasting (PatchTST)	Sequential Memory-Bound	3.8	16.1	15.0	1.07x
Image Classification (ResNet8)	Dense GEMM Acceleration	1.2	54.1	8.1	6.68x
Keyword Spotting (DS-CNN)	Memory-Bound	0.8	14.2	8.0	1.77x
Visual Wake Words (MobileNetV2)	Memory-Bound	1.5	7.0	4.5	1.56x
Anomaly Detection (Autoencoder)	Memory-Bound	0.6	5.8	2.8	2.07x
Image Generation (EDM Diffusion)	Dense GEMM Acceleration	480.0	694.8	180.0	3.86x
Recommendation (NCF)	Sparse Embedding Memory-Bound	120.0	45.5	35.0	1.30x
Reinforcement Learning (MiniGo)	Sequential Search	45.0	51.0	50.0	1.02x